

Containers and Docker

Introduction

Containers and Docker

Agenda

- [08:15 – 08:30] – Setting up Docker on your machines
- [08:30 – 09:00] – Quick intro to Docker
- [09:15 – 12:00] – Follow-along exercises

Preparation

For the exercises:

- [Docker: Overview](#)
- [Docker: Dockerfile reference](#)
- [Docker: Concepts \(basics and building images + containers\)](#)
- [Docker: Get Docker](#)
- [Docker: Install Docker Engine \(can also be done through Docker Desktop\)](#)
- [Docker: Introduction](#)

Exercises

The following exercises will guide you through setting up Docker and using it for your data science projects:

- [Notes and Theory](#) - Understanding Docker concepts
- [Exercise 0: Installation](#) - Install and test Docker
- [Exercise 1: Create a Dockerfile](#) - Build Dockerfile for training
- [Exercise 2: Build the Image](#) - Build and tag your image
- [Exercise 3: Run your Image](#) - Run containers with various options
- [Exercise 4: Push and Pull](#) - Work with registries
- [Exercise 5: Clean up](#) - Remove unused images

Notes and Theory

Prelude

Before getting started with using Docker, it's a good idea to motivate and explain *what* Docker actually is so you don't just type commands and see stuff

happen.

Why?



To avoid the above from happening. Or even shorter -- reproducibility.

Docker essentially allows running code in an isolated and agnostic way from the operating system, making it crucial for reproducible, automated workflows for both normal DevOps and MLOps.

Itemised list for convenience:

- Reproducible and consistent delivery
- CI/CD pipelines
- Portable and collaborative

- Efficient use of resources

What?

Note: Fun fact: Docker is written in Go.

Docker is a way to create a virtual instance of a compute resources. However, unlike virtual machines with their own complete operating system, Docker uses lightweight containers instead.

The key thing is these containers have their own filesystem, dependencies, processes and such, which allows you to run an application in isolation of your operating system.

There are three main components to Docker:

1. Dockerfiles
2. Docker images
3. Docker containers

You will explore each of these components more closely through the upcoming exercises, but a short introduction is in order.

Dockerfiles are a set of instructions that will be used to build a **Docker image**. These instructions include information about the *base image* and the *commands* that will be executed (such as installing packages etc).

A **Docker image** is the output from building the **Dockerfile**. They are immutable, but you can build on top of other images by using them as *base images*.

These images can be tagged and versioned, which also makes it easy to roll back and such to other versions. While we will not touch on this too much, you typically also have image repositories that you can push and pull your images to/from ([Docker Hub](#) being the default).

They also have a layered structure where each command/instruction is essentially cached. That means fast and efficient build times when only making changes in parts of the code.

When you want to run the **Docker image**, it spins it up in a **Docker container** as an isolated runtime environment. Once complete, the container shuts down and releases the resources (compute and storage).

Although they are based on the immutable **Docker image**, it is possible to customize the **Docker container** at runtime. This can also be a way to ensure the same environment for *dev* and *prod* work.

But enough theory for now. Let's start with the exercises!

[Next: Exercise 0 - Installation](#)

Installation

Exercise 0: Installation

As described in the previous section, Docker is really a way to run code in a reproducible way.

Exercise 0: Installation

1. Install Docker Desktop+Engine

If you haven't already, you need to make sure to install [Docker Engine](#). The easiest way is to do it through [Docker Desktop](#). For Windows users, you will also need to install [WSL](#) if you haven't already.

2. Test your Docker installation is working

In the terminal, run `docker run hello-world`

If it works, great! If not, try googling the issue.

A common issue is that the Docker daemon not running. If you installed it with Docker Desktop, make sure that program is running.

[Next: Exercise 1 - Create a Dockerfile](#)

Create a Dockerfile

Exercise 1: Create a training Dockerfile

Note — Learning outcomes By the end of the exercises, we expect you to be able to do the following:

- Write/create a basic Dockerfile that can run a basic DS project
- Explain the workflow and processes for using Docker
- Motivate the typical ways why you would modify Docker containers at runtime
- Describe image repositories and explain their usage

1. Set up a project folder with the training script

Create a new project directory and copy in the training script from `resources/train.py`:

```
mkdir my-docker-project
cp resources/train.py my-docker-project/
cd my-docker-project
```

The script trains a LogisticRegression classifier on the Iris dataset and saves the model to `artifacts/model.pkl`. You will also need a `requirements.txt` listing the packages it depends on.

2. Create train.dockerfile

You can use your favourite tool to do this. Or do:

```
touch path/to/project/train.dockerfile
```

3. Find appropriate base-image

A `python:3.x` image, for example `python:3.11`.

4. Modify example Dockerfile to fit our use-case

Inspired from [this website](#):

```
FROM python:3.11
WORKDIR /usr/local/app
# Install the application dependencies
COPY requirements.txt ./
RUN pip install --no-cache-dir -r requirements.txt
# Copy in the source code
COPY train.py ./
# Setup an app user so the container doesn't run as the root user
RUN useradd app
USER app
CMD ["python", "train.py"]
```

[Next: Exercise 2 - Build the Image](#)

Build the Image

Exercise 2: Build the image

1. Build the train.dockerfile

Normally you just need to run `docker build .`, but since we want to use a specific .dockerfile, we have to modify it slightly:

Hint:

```
docker build -f train.dockerfile .
```

2. Add a meaningful name by version/tag

Just building the image can make it hard to find again.

Adding tags when building the image is typically the easiest way to deal with this. You can also add tags after it has been built but then you need to know the image id.

Hint:

```
docker build -f train.dockerfile -t iris-train:1.0.0 .
```

3. Changing files/fixing errors

It's possible that you need to rebuild the image because of some errors. Can you spot what the error is now?

Hint:

Python version and requirements.txt does not have scikit-learn

Question: why is it faster building it now?

[Next: Exercise 3 - Run your Image](#)

Run your Image

Exercise 3: Run your image

1. Run your latest image

The basic `docker run <image>` is pretty straight-forward. But what is your image id?

Hint to find list of images

```
docker images
```

2. Solving issues with different arguments

[Full list of arguments here](#)

`-i` (`--interactive`) for going "into" the container and run commands interactively.

`-d` (`--detach`) to don't have the process run in your terminal but detached in the background instead.

`-p` (`--publish`)

```
docker run -p 10000:8080 iris-train
```

`-v` (`--volume`) for mounting a directory, allowing you to access files there outside the container.

```
docker run -v ./artifacts:/usr/local/app/artifacts iris-train
```

`-e` (`--env`) is for setting environment variables, which can sometimes be useful whenever using env variables in your code.

3. Check running containers

Checking running containers can be done by `docker ps`. What about

stopped containers?

```
docker ps -a
```

[Next: Exercise 4 - Push and Pull](#)

Push and Pull

Exercise 4: Push and pull

1. Find an interesting **AI/ML image** and pull it

```
docker pull ghcr.io/mlflow/mlflow
```

And tag it: `docker tag ghcr.io/mlflow/mlflow mlflow:latest`

Or run it directly: `docker run ghcr.io/mlflow/mlflow`

Question: Suggested arguments for running this image?

My suggestions:

```
docker run -p 5005:5000 -v $(pwd)/mlruns:/mlruns -e  
MLFLOW_ALLOW_FILE_STORE=true -d mlflow:latest mlflow server --host  
0.0.0.0 --backend-store-uri /mlruns
```

2. Pushing your image

This will require setting up a user that can push to whichever registry you want to push it to. Generally it would follow this structure:

```
docker login <REGISTRY_HOST>:<REGISTRY_PORT>  
docker tag <IMAGE_ID> <REGISTRY_HOST>:<REGISTRY_PORT>/<APPNAME>:<APPVERSION>  
docker push <REGISTRY_HOST>:<REGISTRY_PORT>/<APPNAME>:<APPVERSION>
```

An example would look like:

```
docker login repo.company.com:3456
docker tag 19fcc4aa71ba repo.company.com:3456/myapp:0.1
docker push repo.company.com:3456/myapp:0.1
```

Or in our case with default Docker Hub:

```
docker login --username jeppetk
docker tag 90c46295c455 jeppetk/iris-train:initial
docker push jeppetk/iris-train
```

3. **What do the hashes represent when pushing/pulling?**

It's the different layers. This can be confirmed with the command `docker inspect 90c46295c455`

[Next: Exercise 5 - Clean up](#)

Clean up

Exercise 5: Clean up

1. **Check local images**

```
docker images
```

2. **Remove images**

```
docker rmi IMAGEID
```

[Back to Introduction](#)